
title: "BCA182 Embedded Systems Programming — Laboratory Activity No. 1" subtitle: "Real-Time Multisensor Room Monitoring System" author: "Mindanao State University – Iligan Institute of Technology"

Laboratory Report

Activity: Laboratory Activity No. 1 — Real-Time Multisensor Room Monitoring System

Course: BCA182 Embedded Systems Programming

Platform: STM32F103C8T6 (Blue Pill) + native FreeRTOS + STM32Cube HAL, built with PlatformIO

Simulation: Wokwi

Repository: github.com/Joal0816/BCA182-RoomMonitor

1. Problem and Requirements

1.1 Problem

A room must be monitored continuously for temperature, humidity, ambient light and motion. The device must present one measurement at a time to the user, allow the user to select which measurement is shown, raise an audible alarm when the temperature leaves a safe range, and save power by blanking the display when the room has been unoccupied for a period of time. The firmware must be a genuine concurrent embedded application built on FreeRTOS.

1.2 Functional requirements

ID	Requirement
FR-01	Periodically obtain temperature (DHT22)
FR-02	Periodically obtain humidity (DHT22)
FR-03	Monitor relative ambient-light level (LDR via ADC)
FR-04	Detect motion using the PIR sensor
FR-05	Display one selected measurement at a time (SSD1306)
FR-06	Switch among Temperature, Humidity, Light and Motion with the rotary encoder
FR-07	Activate the buzzer when temperature is outside 18-30 °C
FR-08	Support ACTIVE and INACTIVE activity states
FR-09	Enter INACTIVE after no motion for a set duration
FR-10	Return to ACTIVE when motion is detected

1.3 Wokwi adaptation

The activity is simulated in Wokwi rather than built on physical hardware. The Wokwi project uses an STM32 Blue Pill, a DHT22, a photoresistor, a PIR sensor, an SSD1306 OLED, a KY-040 rotary encoder and a buzzer. `wokwi.toml` loads the ELF produced by `pico run -e bluepill_f103c8`, so the firmware under simulation is the real STM32Cube/FreeRTOS build (not an Arduino sketch).

`diagram.json` wires the peripherals using the Blue Pill's Wokwi pin labels — short designators such as `A0`, `A1`, `B6`, `B7`, `B8`, `A9`, `A10` and the power rails `3V3.1`, `5V.1`, `GND.1`. The pin table in Section 2.1 lists the equivalent STM32 port/pin names (`PA0`, `PB6`, ...) used by the firmware.

2. System Architecture and Design

2.1 Hardware architecture

Component	Interface	STM32 pin
DHT22	1-wire digital	PA1
LDR	ADC1_CH0	PA0
PIR	GPIO / EXTI0	PB0
SSD1306 OLED	I2C1	PB6 (SCL), PB7 (SDA)
KY-040 encoder	GPIO / EXTI2, EXTI4	PA2 (CLK), PA3 (DT), PA4 (SW)
Buzzer	TIM4_CH3 PWM	PB8
Status LED	GPIO	PC13
USART1	115200 8N1	PA9 (TX), PA10 (RX)

2.2 Software architecture

The firmware separates three concerns:

1. **Drivers** (`src/app/hal`, `src/drivers`) — thin wrappers over the HAL for each peripheral.
2. **Logic** (`src/app/logic`) — hardware-independent decision logic (`temperature`, `state_machine`, `display_page`) plus the buzzer alarm policy.
3. **Tasks** (`src/app/tasks`) and `src/main.c` — the FreeRTOS application.

`main()` follows the STM32 startup convention and delegates to `app_main()`, the application entry point required by the activity. `app_main()` initializes the HAL, creates the RTOS objects and tasks, and starts the scheduler.

Peripheral clocks and GPIO alternate functions are configured in `src/stm32f1xx_hal_msp.c` through the STM32Cube MSP callbacks (`HAL_UART_MspInit`, `HAL_I2C_MspInit`,

`HAL_ADC_MspInit`, `HAL_TIM_PWM_MspInit`). These callbacks enable `USART1`, `I2C1`, `ADC1` and `TIM4` and map `PA9/PA10`, `PB6/PB7`, `PA0` and `PB8`; without them the HAL would access unlocked peripherals and produce no UART output.

2.3 Subsystem decomposition

- **Acquisition** — `SensorTask` reads DHT22 + LDR once per second and publishes a single `SensorData_t` sample to two queues.
- **Decision** — `AlarmTask` evaluates temperature status; `StateTask` runs the ACTIVE/INACTIVE machine.
- **Presentation** — `DisplayTask` owns the OLED and renders the selected page.
- **Input** — `InputTask` translates encoder events into page changes.

2.4 State machine

The system has two states:

- **ACTIVE** — OLED on, sensors and alarm processing normal, navigation active.
- **INACTIVE** — OLED switched off, display operations suppressed, motion detection still operational.

The transition to INACTIVE occurs after `INACTIVE_TIMEOUT_MS` (15 s) without motion; any motion returns the system to ACTIVE and resets the timer.

3. FreeRTOS Architecture

3.1 Task table

Task	Responsibility	Trigger / Period	Priority	IPC	Typical blocked condition
<code>SensorTask</code>	Read DHT22 + LDR	1 s periodic	2	Queues (<code>alarm_queue</code> , <code>display_queue</code>)	<code>vTaskDelayUntil()</code>
<code>AlarmTask</code>	Evaluate alarm, drive buzzer	Sensor update	2	Queue (<code>alarm_queue</code>)	<code>xQueueReceive()</code>
<code>StateTask</code>	ACTIVE/INACTIVE state machine	250 ms poll / motion event	2	Event group	<code>xEventGroupWaitBits()</code>
<code>InputTask</code>	Rotary-encoder navigation	Encoder EXTI notification	3	Notification + <code>display_page_queue</code>	<code>ulTaskNotifyTake()</code>
<code>MotionTask</code>	Monitor PIR		3		<code>ulTaskNotifyTake()</code>

Task	Responsibility	Trigger / Period	Priority	IPC	Typical blocked condition
		PIR EXTI / 500 ms refresh		Event group (EVENT_MOTION_BIT)	
DisplayTask	Own and manage the OLED	100 ms refresh	1	Queues + event group read	xQueueReceive()

3.2 Priority justification

- **InputTask and MotionTask (3)** must respond to a user or environmental event quickly; latency here is directly perceptible (laggy navigation, delayed reactivation). They block on notifications, so the higher priority costs no CPU while idle.
- **SensorTask, AlarmTask and StateTask (2)** are periodic or derived. They tolerate milliseconds of jitter but must run before the display so that fresh data and safety decisions are available to render.
- **DisplayTask (1)** is cosmetic; it may lag without affecting correctness and must never preempt the producers.

If the periodic workers outranked the event tasks, input responsiveness would degrade; if the display outranked them, sampling and alarm latency would increase (Fault Experiment 2).

3.3 Task states

A task is **Blocked** whenever it is waiting on a delay, queue, event group or notification, and yields the CPU. When the wait completes it becomes **Ready**; the scheduler moves it to **Running** when it is the highest-priority ready task. No task contains an uncontrolled busy loop: every task performs finite work and then blocks.

3.4 Communication and synchronization

Primitive	Type	Producer	Consumer	Purpose
alarm_queue	Queue (depth 5)	SensorTask	AlarmTask	Every sample for alarm evaluation
display_queue	Queue (depth 5)	SensorTask	DisplayTask	Every sample for display
display_page_queue	Queue (depth 1, overwrite)	InputTask	DisplayTask	Latest selected page
event_group	Event group	MotionTask, StateTask	StateTask, DisplayTask	Motion and ACTIVE-state events
uart_mutex	Mutex	all tasks	USART1	Serialize multi-byte UART writes

Primitive	Type	Producer	Consumer	Purpose
Task notification	Direct-to-task	PIR / encoder ISR	MotionTask / InputTask	Fast event wake-up

Event-group bits. `EVENT_MOTION_BIT` is set by MotionTask and consumed by StateTask, which uses `xEventGroupWaitBits()` to block instead of polling. `EVENT_STATE_ACTIVE_BIT` is produced by StateTask and read by DisplayTask to switch the OLED on or off.

3.5 `vTaskDelayUntil()` versus `vTaskDelay()`

`vTaskDelay()` delays relative to the instant it is called, so the time spent doing work before the call is added to each period and the sampling instant drifts. `vTaskDelayUntil()` takes an absolute wake time that is advanced by exactly one period per iteration, so the sampling instant stays fixed regardless of how long a read takes. For periodic sensor sampling, `vTaskDelayUntil()` prevents accumulated drift and is therefore used in SensorTask.

4. Implementation

4.1 Modular organization

The application is not contained in `main.c`. Drivers, pure logic and tasks each live in their own translation unit. `main.c` performs hardware initialization, creates the RTOS objects, creates the tasks and starts the scheduler.

4.2 Key decisions

- **Two sensor queues.** An early design used one queue with two consumers (AlarmTask and DisplayTask). Because a FreeRTOS queue delivers each item to exactly one consumer, the two tasks competed and each reader missed samples. The final design publishes every sample to two independent queues so both consumers receive every reading.
- **Single owner of the state machine.** `StateMachine_t` is read and written only by StateTask. Other tasks learn the state through `EVENT_STATE_ACTIVE_BIT`, removing a shared-mutable-state race.
- **Pure logic.** `EvaluateTemperature()`, `StateMachine_Update()` and `DisplayPage_Next()/Previous()` have no HAL or FreeRTOS dependency, so they are compiled and tested on the host PC.
- **NAN on sensor fault.** A failed DHT22 read is published as `NAN`; AlarmTask ignores `NAN`, so a sensor failure cannot raise a false temperature alarm.
- **Critical sections for the encoder.** `Encoder_GetDelta()` reads and clears the position inside a critical section so an ISR cannot lose a detent.

4.3 Periodic sampling

```
TickType_t lastWakeTime = xTaskGetTickCount();
for (;;) {
    /* read DHT22 + LDR, publish sample */
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(SENSOR_READ_PERIOD_MS));
}
```

5. Verification and Testing

5.1 Unit tests

Run with `pio test -e native`. The tests compile and link the real logic from `src/app/logic/`, so they fail if production behaviour changes.

Suite	Tests	Coverage
test_temperature	15	Below / exactly at / above both thresholds; status strings; alarm predicate
test_navigation	10	Forward and reverse transitions, both wraparounds, cancellation
test_state_machine	8	ACTIVE↔INACTIVE, timeout boundary, motion reset, sustained motion
Total	33	33/33 pass

5.2 Functional verification (Wokwi)

The ten required functional tests (FT-01 ... FT-10) and their procedures are listed in `docs/functional-verification.md`. The observed results are recorded in that file from a live Wokwi session; a test is only marked PASS once its actual behaviour has been recorded.

5.3 Deliberate fault experiments

Three faults were injected and reverted, as required (details in `docs/fault-experiments.md`):

1. **Remove blocking** — deleting the periodic delay caused CPU starvation and a frozen display; restored `vTaskDelayUntil()`.
2. **Change priority** — raising DisplayTask to priority 4 degraded sampling and alarm latency; restored priority 1.
3. **Remove mutex** — removing UART protection produced interleaved log lines; restored the mutex.

6. Static Code Analysis

`pio check -e bluepill_f103c8` reports **0 HIGH, 0 MEDIUM and 76 LOW** findings.

Category	Count	Interpretation
<code>unusedFunction</code>	54	cppcheck analyses each translation unit in isolation; functions called from other files or via function pointers (task entry points, IRQ handlers) are reported as unused. Verified reachable.
<code>constParameterPointer</code>	1	Read-only accessor parameters that could be <code>const</code> ; most were corrected during this review, one remains.

No finding indicates a correctness or safety defect. The full findings table and corrective actions are in `docs/static-analysis.md`.

7. Engineering Discussion

7.1 Trade-offs

- **One queue per consumer** costs a small amount of RAM but removes the competing-consumer data-loss bug; on a 20 KB device this is the right trade.
- **Priority 2 for three periodic tasks** keeps the design simple; time slicing guarantees each a share of the CPU.
- **Polled state timeout (250 ms)** avoids a dedicated timer task while keeping the state transition responsive.

7.2 Limitations

- LDR readings are raw 12-bit values, not calibrated lux.
- SRAM usage is high (the FreeRTOS heap and task stacks occupy most of the 20 KB), leaving limited headroom for new features.
- The DHT22 one-wire read masks interrupts briefly.
- Wokwi's peripheral fidelity differs from real hardware.

7.3 Debugging notes

The main difficulties encountered were the competing-consumer queue (samples disappearing under load), the DHT22 timing (resolved with DWT cycle-counting inside a critical section) and the state-machine race (resolved by centralizing state in `StateTask`). FreeRTOS stack-overflow and malloc-failed hooks provided early warning during development.

8. Conclusion

The system implements all ten functional requirements using six concurrent FreeRTOS tasks, explicit priorities, `vTaskDelayUntil()` periodic sampling, queues, a mutex, an event group and task notifications, plus an ACTIVE/INACTIVE state machine. The hardware-independent logic is verified by 33 passing host unit tests, and static analysis reports no correctness defects.

What was learned: how to design around FreeRTOS scheduling semantics (blocking, priorities and task states), why each synchronization primitive must have a real producer and consumer, and how separating pure logic from hardware makes an embedded application testable. Future improvements include connectivity, data logging and tighter RAM budgeting.

Appendix A — FreeRTOS Task Table (summary)

Task	Responsibility	Trigger / Period	Priority	IPC	Typical blocked condition
SensorTask	Measurements	1 s	2	Queue	Delay
DisplayTask	OLED	event / 100 ms refresh	1	Queue + event group	Waiting for data
InputTask	Encoder	event	3	Notification + queue	Wait
MotionTask	PIR	event / 500 ms	3	Event group	Wait
AlarmTask	Alarm logic	sensor update	2	Queue	Waiting for data
StateTask	System state	250 ms / event	2	Event group	Waiting for event

Appendix B — Requirements Traceability

Requirement	Implementation	Verification
FR-01 Temperature	SensorTask, DHT22_Read	test_temperature, FT-01
FR-02 Humidity	SensorTask, DHT22_GetHumidity	FT-02
FR-03 Light	SensorTask, LDR_Read	FT-03
FR-04 Motion	MotionTask, PIR_GetState	FT-08, FT-10
FR-05 Display	DisplayTask	FT-01-FT-03
FR-06 Navigation	InputTask, DisplayPage_Next/Previous	test_navigation, FT-04, FT-05
FR-07 Alarm		

Requirement	Implementation	Verification
	AlarmTask, EvaluateTemperature, Alarm_Update	test_temperature, FT-06, FT-07
FR-08 Activity state	StateTask, StateMachine_Update	test_state_machine, FT-08
FR-09 Inactivity	StateTask, StateMachine_Update	test_state_machine, FT-09
FR-10 Reactivation	StateTask	test_state_machine, FT-10